

This section consolidates learning outcomes and provides guidance for applying custom agent concepts to real-world development scenarios. Participants review key takeaways including agent role specialization, stakeholder-driven development gates, file-based collaboration transparency, and security boundary enforcement, while exploring practical applications for small teams, large organizations, client projects, and compliance environments.

Wrap-up

Key Takeaways

- Custom agents mirror real team roles** - Product Manager, Developer, QA Tester
- UAT user drives team decisions** - stakeholder feedback shapes agent behavior
- Simple, clear requirements** lead to better outcomes than complex specifications
- File-based collaboration** creates transparency and accountability
- Permission boundaries** ensure agents stay in their lanes like real team members
- Security considerations** - Current allowedPaths model has limitations; agents may create alternative files to bypass restrictions

Real-World Team Benefits

- **Clear role separation** prevents confusion and overlap
- **Stakeholder involvement** ensures user needs are met
- **Quality gates** through testing and UAT approval
- **Audit trail** of decisions and changes through shared files
- **Scalable process** that works for simple or complex projects

Real-World Applications

- **Small teams:** Each person can use agents to handle different aspects of their work
- **Large organizations:** Agents can represent different departments or stakeholders
- **Client projects:** UAT agent can represent client feedback and approval process
- **Compliance environments:** Agents ensure proper review and approval workflows

Bonus

Advanced Implementation Ideas

Business Bob:

Review the advanced features and testing results. Create user acceptance criteria in shared/decisions.md for:

- Animation smoothness standards
- Mobile interaction guidelines
- Accessibility requirements
- Performance benchmarks for our modern todo app

Developer Dave:

Implement advanced modern features:

- Create a dark/light mode toggle component
- Add smooth transitions for task state changes
- Implement a progress visualization component
- Create responsive grid layout for different screen sizes

Show the code structure and update shared/status.md with the modern patterns used.

QA Quinn:

Test the advanced features Dave implemented:

- Validate dark/light mode accessibility (color contrast ratios)
- Test animations on different devices and browsers
- Verify responsive behavior across screen sizes
- Check keyboard navigation for all interactive elements

Report findings in shared/test-results.md with specific modern app testing metrics.

Extension Ideas

- **Add more roles:** DevOps Engineer, Security Reviewer, Technical Writer
- **Complex features:** User authentication, data persistence, API documentation
- **CI/CD integration:** Agents that trigger builds and deployments
- **Multi-project:** Scale to multiple applications with shared agents

Integration with Existing Workflows

- **Project management:** Connect agents to Jira, Trello, or GitHub Issues
- **Documentation:** Agents that update wikis and technical documentation
- **Monitoring:** Agents that respond to alerts and system issues
- **Code review:** Agents that participate in pull request reviews

Next Steps

- Apply this pattern to your real projects
- Create agents that match your actual team structure
- Use UAT feedback loops to improve requirements quality
- Scale up complexity gradually as teams get comfortable
- Integrate with existing project management and approval processes

This workshop demonstrates how custom agents can replicate real-world team dynamics while maintaining quality, accountability, and stakeholder satisfaction.